Examen Final

L'examen dure 1 heure 50 minutes et comporte 5 exercices. Les documents sont autorisés, mais limités à 10 feuilles recto-verso par personne. Aucun appareil numérique n'est autorisé, donc, de façon non-exhaustive : pas d'ordinateur, pas de téléphone, pas de montre connectée, etc. Pas de calculatrice non plus.

Exercice 1 (Degrés sortants et entrants)

Étant donné un graphe orienté $\vec{G}$ on définit le graphe non-orienté G de la façon suivante : $V(G) = V(\vec{G})$ et deux sommet x et y sont adjacents dans G si est seulement xy ou yx est un arc de $\vec{G}$.

Expliquez comment calculer la liste d'adjacence de G étant donné celle de $\vec{G}$. Pas besoin de donner un pseudo-code, mais justifier la complexité.

Exercice 2 (Dijkstra)

Exécuter l'algorithme de Dijkstra au graphe de la Figure 1 avec comme source A.

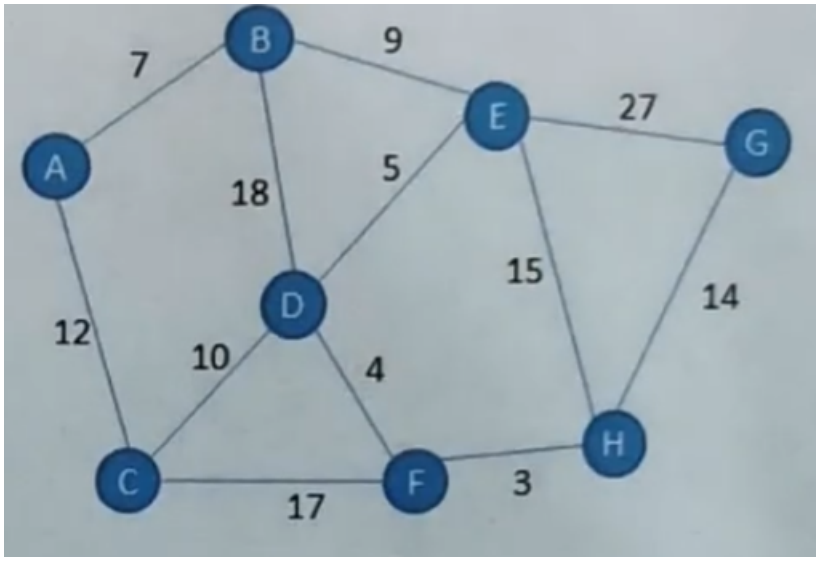


FIGURE 1 – Graphe de l'exercice 2

Exercice 3 (Fermeture transitive)

Dans cette exercice on considère des graphes dirigés tel que, pour toute paire de sommets x, y, il y a au plus un arc allant de x vers y, mais il peut y avoir un arc de x vers y et un autre de y vers x.

Étant donné un graphe dirigé $G = (V, A)$ où $V = \{v_1, v_2, \ldots, v_n\}$, la fermeture transitive de G est le graphe $G^* = (V, A^*)$, où $v_i v_j \in A^*$ si et seulement si il existe un chemin de v_i à v_j dans G.

Question 1 Dessiner la fermeture transitive du graphe orienté suivant : $V = \{v_1, v_2, v_3, v_4, v_5\}$, $A = \{v_1v_2, v_1v_5, v_2v_3, v_3v_4, v_5v_4\}$.

Question 2 Combien d'arcs a la fermeture transitive du chemin sur n sommets P_n (c'est à dire le graphe dirigé sur les sommets $v_1, \ldots, v_n$ avec les arcs $\{v_1v_2, v_2v_3, \ldots, v_{n-1}v_n\}$).

Question 3 Quel est la fermeture transitive d'un graphe dirigé G fortement connexe [1].

Question 4 Combien d'arcs a la fermeture transitive du cycle sur n sommets C_n (c'est à dire le graphe dirigé sur les sommets $v_1, \ldots, v_n$ avec les arcs $\{v_1v_2, v_2v_3, \ldots, v_{n-1}v_n, v_nv_1\}$).

On définit $TR(i, j, k)$ comme étant égal à 1 s'il existe un chemin de v_i vers v_j n'utilisant que des sommets de $\{v_i, v_j\} \cup \{v_1, \ldots, v_k\}$.

Question 5 Donnez un algorithme qui calcule la fermeture transitive d'un graphe G en utilisant de la programmation dynamique. On pourra utiliser les sous-problèmes TR définit ci-dessus.

On vous demande de : 1- Traiter les cas de bases, 2- explicitez la formule de récurrence, 3- Donnez un pseudocode (sans oublier les cas de bases).

ATTENTION : interdit d'utiliser des algorithmes étudiés en cours, c'est à dire qu'on ne peux pas dire "j'exécute Dijkstra" ou autre (on peut par contre s'inspirer des idées vues dans des algos).

Exercice 4 (Tas-max)

Question 1 Dessinez tous les tas-max contenant les éléments $\{5, 7, 9\}$, puis les éléments $\{3, 7, 11, 14\}$.

Question 2 On a vu dans le cours qu'un tas-max peut-être implémenté dans un tableau. Représentez le tableau suivant comme un arbre : $[27, 18, 6, 14, 2, 8, 4, 12, 3]$. Est-ce un tas-max ?

Question 3 Faites un algorithmes testant si un arbre binaire est un tas-min. Vous expliquerez l'algo, ferez un pseudo-code, et donnerez sa complexité.

Question 4 En utilisant l'algorithme vue en cours, insérer un élément de clé 17 dans le tas-max de la Figure 2. Répondre en représentant le nouveau tas.

Question 5 En utilisant l'algorithme vue en cours,, et en repartant du tas initial, retirer l'élément de clé maximale. Répondre en représentant le nouveau tas.

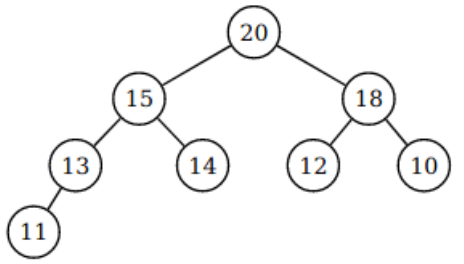


FIGURE 2 – Tas-max de l'exercice 5

Exercice 5 (Labyrinthe – Python)

Vous êtes dans un labyrinthe. Vous devez retrouver *le plus court chemin* du point de départ `S` jusqu'à la sortie `E`. Pour ce faire, on veut déterminer la séquence de déplacements à faire pour rejoindre la sortie. Le labyrinthe est représenté comme une grille de caractères, où chaque case de la grille peut être :

— `#` : un mur (non franchissable),

— `.` : une case libre donc franchissable,

— `S` : le point de départ (start),

1. On rappelle qu'un graphe est fortement connexe si pour toute paire de sommets u, v, il existe un chemin de u vers v.

— `E` : la sortie (end).

On peut se déplacer **uniquement dans les 4 directions orthogonales** : `U` = haut (up), `D` = bas (down), `L` = gauche (left), `R` = droite (right). Voici un exemple de labyrinthe suivant cette représentation :

```
exemple_labyrinthe = [
    "######",
    "#.S..#",
    "#.##.#",
    "#....#",
    "###E##"
]
```

Une façon de s'échapper de ce labyrinthe en faisant un minimum de déplacements serait de faire les déplacements `"LDDRRD"`. Notez que pour certains labyrinthes il est possible d'avoir plusieurs chemins ayant une longueur minimale : retourner l'un d'entre eux est suffisant (dans l'exemple, `"RRDDLD"` marche également).

Question 1 Pourquoi l'utilisation de l'algorithme BFS est particulièrement adaptée pour ce problème du plus court chemin ? Expliquer.

Question 2 Écrire une fonction Python `trouver_debut_sortie(labyrinthe)` prenant en paramètre un labyrinthe possédant la structure décrite plus haut, et retournant deux tuples Python correspondant à la position (les coordonnées) du départ et de la sortie dans le labyrinthe. *Rappels/Indice : Pour retourner deux éléments a et b, on écrit* `return a, b`. *On rappelle aussi qu'en Python,* ***une chaîne de caractères peut être vue comme une liste de caractères****. Ainsi, le labyrinthe n'est rien d'autre qu'une liste de listes. . .*

Question 3 Compléter le code Python de la fonction `plus_court_chemin(labyrinthe)`, (**voir page suivante**), partout où des champs « `____` » apparaissent. Quelques indications :

— **Vous devez répondre dans votre copie et non sur le sujet** : vous pouvez pour ce faire écrire le numéro de la ligne suivi de la ligne complétée.

— Le nombre de « `_` » est choisi aléatoirement et ne reflète donc pas le nombre de caractères attendu.

— Des précisions sont données sous la forme de commentaires Python pour aider à la compréhension.

SUITE A LA PROCHAINE PAGE

```python
from collections import deque

def plus_court_chemin(labyrinthe):
    n, m = len(labyrinthe), len(labyrinthe[0])
    debut, sortie = trouver_debut_sortie(labyrinthe)

    deja_visitees = [[False]*m for _ in range(n)] # Structure de donnees pour garder trace des cases deja visitees
    deplacements = {}  # Dictionnaire gardant trace des differents deplacements realises : (x, y) -> (px, py, direction)

    # On initialise la file et on visite la case de depart
    file_ = deque() # file est un mot reserve en Python, on utilise file_
    file_.append(debut)
    deja_visitees[debut[0]][debut[1]] = True

    # Les deplacements possibles sont les directions orthogonales
    directions = [(-1,0,'U'), (1,0,'D'), (0,-1,'L'), (0,1,'R')]

    while file_:
        x, y = file_.popleft()
        if (x, y) == sortie:
            break
        for dx, dy, lettre in directions:
            nx, ny = x + dx, y + dy
            if _________________________: # Condition(s) pour ne pas chercher en dehors du tableau
                if _________________________: # Condition(s) pour pouvoir visiter la case (nx, ny)
                    deja_visitees[___][___] = _____
                    file_.append(_____)
                    deplacements[(nx, ny)] = (x, y, lettre)

    # Reconstruction du chemin en partant de la sortie et en remontant jusqu'au debut
    chemin = []
    case_courante = sortie
    while case_courante != debut:
        px, py, lettre = ____________
        chemin.append(__________)
        case_courante = (px, py)

    return ''.join(reversed(chemin)) # On retourne les deplacements dans le bon ordre
```

Question 4 Quelle est la complexité temporelle de cet algorithme ? Justifier.